

Numerical Python

the `numpy` module



Numerical Python: the **numpy** module

- ***Numpy*** is a python package, stands for numerical python
- **Operations using **numpy**:**
 - Mathematical and logical operations on **arrays**
 - Operations related to **linear** algebra
 - Built-in functions for **random** number generation
- **Website:** <https://numpy.org>

The `numpy` module: `ndarray`

- Most important object defined in NumPy is an N-dimensional array type called `ndarray`
- It describes the collection of items of the same type
- Items in the collection can be accessed using a zero-based index

```
import numpy as np

L = [1, 2, 3]
a = np.array(L)
print(a)

L = [[1, 2, 3], [4, 5, 6]]
a = np.array(L)
print(a)
```

Output:

```
[1 2 3]
[[1 2 3]]
```

The `numpy` module: `ndarray` (cont'd)

- Unlike Python lists, you can specify the data-type of the items in a `numpy` array.
- Python automatically casts all the items to the specified *dtype*
- Once you specify the *dtype*, mixed types will not be allowed

```
import numpy as np

a_mixed = np.array([1, 'CSC243'])
print(a_mixed)

a_float = np.array([1, 2, 3 ], dtype=float)
print(a_float)
```

Output:

['1' 'CSC243']

[1. 2. 3.]

The `numpy` module: `ndarray` (cont'd)

- Computing the length of the `ndarray` in `numpy` is done via the *`shape`* attribute

```
import numpy as np

L = [[1, 2, 3], [4, 5, 6]]
a = np.array(L)
print(a.shape)
```

Output:
(2, 3)

The `numpy` module: `ndarray` (cont'd)

- *Numpy* arrays are created: from existing data (as seen previously), as empty arrays, as zeros, as ones.
- Examples:

```
import numpy as np

a = np.empty((2,2), dtype=float)
print(a)
a = np.zeros((2,2))
print(a)
a = np.ones((3,))
print(a)
a = np.ones((3,), dtype=int)
print(a)
```

Output:

```
[[0. 0.]
 [0. 0.]]
```

```
[[0. 0.]
 [0. 0.]]
```

```
[1. 1. 1.]
```

```
[1 1 1]
```



The `numpy` module: ranges

In `numpy` we can create ranges of data through the following:

- `np.arange(start, stop, step, dtype)`: returns an ndarray object containing evenly spaced values within a given range
- `np.arange(stop, dtype)`: returns an ndarray object containing evenly spaced values from 0 till stop-1
- `np.linspace(start, stop, num, dtype)`: instead of step size, the number *num* of evenly spaced values in the interval is specified.

The numpy module: ranges (cont'd)

- One advantage of using `numpy.arange` instead of `range` is that the step-size can be of type `float` in this case
- Examples:

```
import numpy as np

r = np.arange(1.5, 2, 0.1)
print(r)
r = np.arange(5)
print(r)
r = np.linspace(10, 50, 5)
print(r)
```

Output:

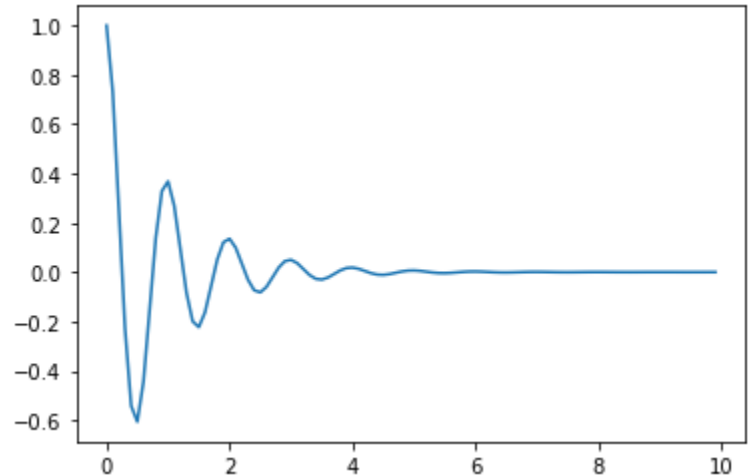
```
[1.5 1.6 1.7 1.8 1.9]
[0 1 2 3 4]
[10. 20. 30. 40. 50.]
```

The numpy module: math functions

- Mathematical functions can also be accessed from the numpy module (sin, cos, exp, ceil, floor, etc.)
- Advantage of using numpy function is that we can pass a full array as input
- **Example:** plotting $e^{-x}\cos(2\pi x)$ for $x = [0, 0.1, 0.2, \dots, 9.9]$

```
import matplotlib.pyplot as plt
import numpy as np
from numpy import pi

X = np.arange(0,10,0.1)
Y = np.exp(-X)*np.cos(2*pi*X)
plt.plot(X, Y)
```



The numpy module: statistical operations

- Min and max through `np.amin(a, axis=)` and `np.amax(a, axis=)`
- The `axis` keyword specifies the dimension over which min/max is computed
- If not specified, the min/max will be computed over the whole array

For `axis=0`, this means that we apply a function along each “column”, or all values that occur vertically.
For `axis=1`, this means that we apply a function along each “row”, or all values horizontally.

```
import numpy as np

a = np.array([[1,2], [3,4], [5,6]])
print("a=")
print(a)
print("min=", np.amin(a))
print("minima (axis=0):", np.amin(a, axis=0))
print("minima (axi=1):", np.amin(a, axis=1))
```

```
a=
[[1 2]
 [3 4]
 [5 6]]
min= 1
minima (axis=0): [1 2]
minima (axi=1): [1 3 5]
```

The numpy module: statistical operations (cont'd)

- To compute mean, median, standard deviation and variance use `np.mean()`, `np.median()`, `np.std()`, `np.var()`, respectively
- All these functions also admit an axis keyword parameter like `np.amin`

```
import numpy as np

# grades for 3 students, on 2 assignments
a = np.array([[90, 92], [81, 97], [70, 82]])
avg0 = np.average(a, axis=0)
print("averages, per assignment (axis=0)")
print(avg0)
avg1 = np.average(a, axis=1)
print("averages, per student (axis = 1):")
print(avg1)
print("class average:", np.average(a))
```

```
averages, per assignment (axis=0)
[80.33333333 90.33333333]
averages, per student (axis = 1):
[91. 89. 76.]
class average: 85.33333333333333
```

Generating random numbers

- **import `numpy.random` as `rand`**
- **Basic random functions:**
 - `rand.uniform(x,y)`: generates a uniformly random float in the real interval $[x,y]$ (default $x=0$ and $y=1$)
 - `rand.randint(a,b+1)`: generates a uniformly random integer between a and b inclusive
- **Multiple runs give different random numbers**